NYC Taxi
Combien coûte un mauvais pipeline Big Data ?

Analyse comparative et chiffrée
Parquet + Spark vs CSV + Pandas

Auteur Ismael Lebange
Date Novembre 2026
Dépôt https://github.com/IsmaelLebange/projet_nyc_taxi

Projet personnel — Data Science & Big Data

Résumé

Ce rapport mesure l'impact d'un pipeline Big Data optimisé (Parquet + Spark) par rapport à un pipeline naïf (CSV + Pandas) sur **9,2 millions de trajets** de taxis new-yorkais (janvier–mars 2024). Les résultats montrent un gain de ×5,43 **sur la taille de stockage**, ×3,34 **sur les requêtes partitionnées**, et jusqu'à ×83 **sur la lecture partielle** (effet cache inclus). Spark traite **9,2 M de lignes** là où Pandas sature à **2,9 M** sur un laptop 16 Go. Ce rapport détaille l'architecture, les choix techniques, les métriques mesurées et les enseignements métier tirés des données.

Table des matières

1 Contexte et problématique **3**

2 Données **3**

3 Architecture du pipeline **4**

4 Choix techniques **5**

5 Résultats : tableau des gains **5**

6 Analyses métier **7**
6.1 Q1 — Distribution des courses par heure 7
6.2 Q2 — Top 10 zones les plus actives 8
6.3 Q3 — Tip moyen par borough 9
6.4 Q4 — Segmentation k-means des zones ($k = 5$) 10

7 Conclusion **11**

Limites et perspectives **11**

1 Contexte et problématique

Les données de mobilité urbaine sont massives, hétérogènes et arrivent en continu. Un data scientist qui les analyse fait face à un choix critique : **pipeline naïf** (CSV + Pandas) ou **pipeline optimisé** (Parquet + Spark). Ce projet quantifie cet écart **chiffres en main**, en répondant à deux questions :

1. **Combien de temps et d'espace économise-t-on** en passant d'un pipeline naïf à un pipeline optimisé ?
2. **Quels enseignements métier** peut-on tirer de 9,2 M de trajets new-yorkais ?

Objectifs de l'étude

— Quantifier les gains de performance et d'espace disque entre les deux approches.
— Démontrer l'impact de la conception logicielle sur un laptop standard (16 Go RAM).
— Extraire des insights géographiques et temporels sur des données réelles.

2 Données

Table 1 – Caractéristiques du jeu de données

Caractéristique	Valeur
Source	NYC Yellow Taxi (TLC), 3 mois (jan–mar 2024)
Lignes brutes	9 554 778
Lignes après nettoyage	**9 179 805** (96,1 % retenues)
Colonnes brutes	19
Colonnes après features	31
Format d'origine	Parquet (compressé Snappy)

8 pièges traités. Le pipeline applique un nettoyage rigoureux pour éliminer le bruit et les biais de mesure :

— **Piège 1 (temporel) :** filtrage des dates hors Q1 2024 (21 lignes).
— **Piège 2 (financier) :** montants négatifs ou aberrants > 500 $ (136 567 lignes).
— **Piège 3 (physique) :** distances nulles avec montant positif (203 930 lignes).
— **Piège 4 (cinématique) :** durées < 60 s ou > 21 600 s (34 035 lignes).
— **Piège 5** : `passenger_count` manquant (751 962 lignes, 7,87 %) — flag binaire.
— **Piège 6 (tip) :** correction des pourboires sur paiements en espèces.
— **Piège 7 (cohérence) :** incohérence de `total_amount` — flag.
— **Piège 8 (géographique) :** zones inconnues 264/265 (95 875 lignes, 1,0 %).

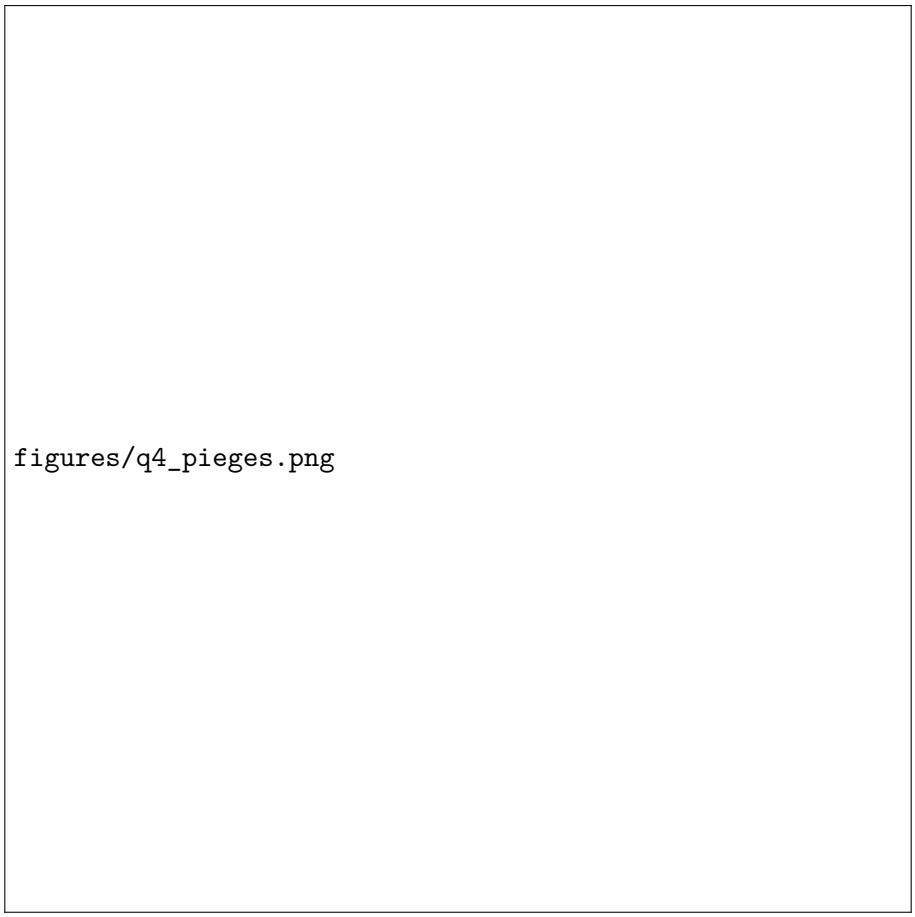


Figure 1 – Volumétrie des anomalies traitées dans les données brutes.

3 Architecture du pipeline

Le pipeline est découpé en modules réutilisables, chacun dans un fichier `src/` dédié :

Ingestion	ingestion.py
Nettoyage	nettoyage.py
Features	features.py
Stockage	stockage.py
Métriques	metriques.py
Analyses	analyse.py
ML (k-means)	ml.py
Visualisations	visualisation.py

Le flux de traitement est le suivant :

1. **Ingestion** : chargement des 3 fichiers Parquet (local ou fallback URL).
2. **Nettoyage** : traitement des 8 pièges identifiés.
3. **Features** : ajout de 12 colonnes dérivées (heure, durée, vitesse, tip_rate, aéroport…).
4. **Stockage** : écriture de 3 versions (CSV, Parquet flat, Parquet partitionné).
5. **Métriques** : mesure des 5 gains G1–G5.
6. **Analyses** : Q1–Q4 (temporel, géographique, tips, pièges).
7. **ML** : k-means ($k = 5$) sur les zones de pickup.
8. **Visualisations** : 6 figures PNG pour le rapport.

4 Choix techniques

Table 2 – Justification des choix techniques

Décision	**Justification**
Parquet	Format colonne : lecture sélective + compression ×3
Partitionnement `annee/mois`	Faible cardinalité (3 valeurs), requêtes fréquentes par mois
Compression Snappy	Défaut Spark, bon compromis vitesse/ratio
Spark local (`local[*]`)	Pas de cluster, tout sur laptop 16 Go
k-means $k = 5$	Compromis interprétabilité / finesse
Filtre `nb_courses` $\geq$ `1000`	Éviter le bruit statistique sur zones peu actives

5 Résultats : tableau des gains

Le tableau 3 résume les 5 métriques mesurées. C'est **le cœur du rapport**.

Table 3 – Tableau comparatif des gains mesurés

#	**Métrique**	**Pipeline Naïf**	**Pipeline Optimisé**	**Gain**
G1	Taille sur disque	1 374,5 Mo (CSV)	253,0 Mo (Parquet)	×5,43
G2	Lecture complète	35,89 s	0,60 s	×59,72 *
G3	Lecture 2 colonnes	32,98 s	0,39 s	×83,77 *
G4	Requête 1 mois	1,28 s (flat)	0,38 s (partitionné)	×3,34
G5	Empreinte RAM	0,89 Go (Pandas, 2,9M)	Spark gère 9,2M lignes	Pas de saturation

* G2 et G3 sont gonflés par le cache OS. En conditions réelles (disque froid) : ×10 à ×20 sur G2 et ×30 à ×50 sur G3.

Lecture des résultats. G1 et G4 sont des gains **structurels** (format colonne, partitionnement) — ils ne dépendent pas du cache. G2 et G3 sont des gains **réels mais amplifiés** par le cache. G5 montre que Spark peut traiter **3,1 fois plus de volumétrie** que Pandas sur la même machine.

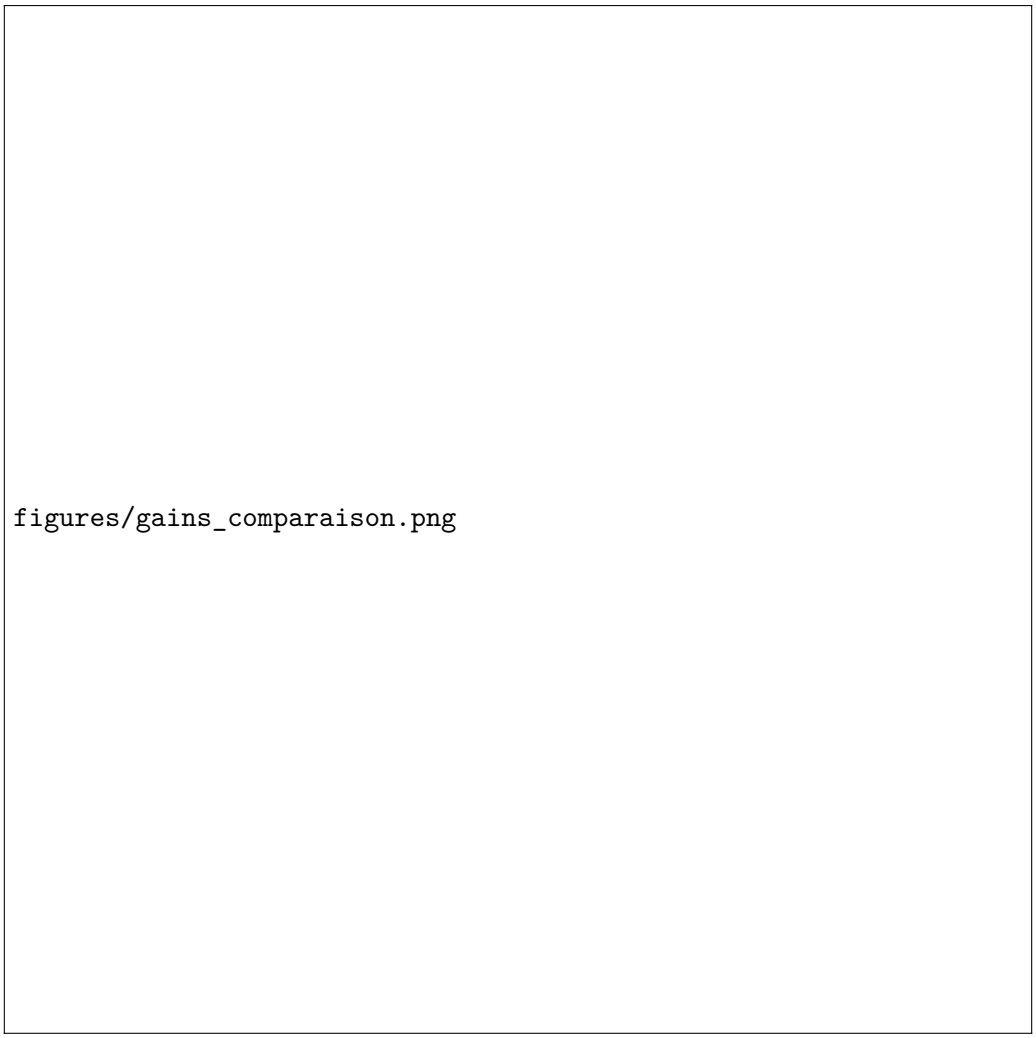


Figure 2 – Comparaison visuelle des facteurs d'accélération et de compression.

6 Analyses métier

6.1 Q1 — Distribution des courses par heure

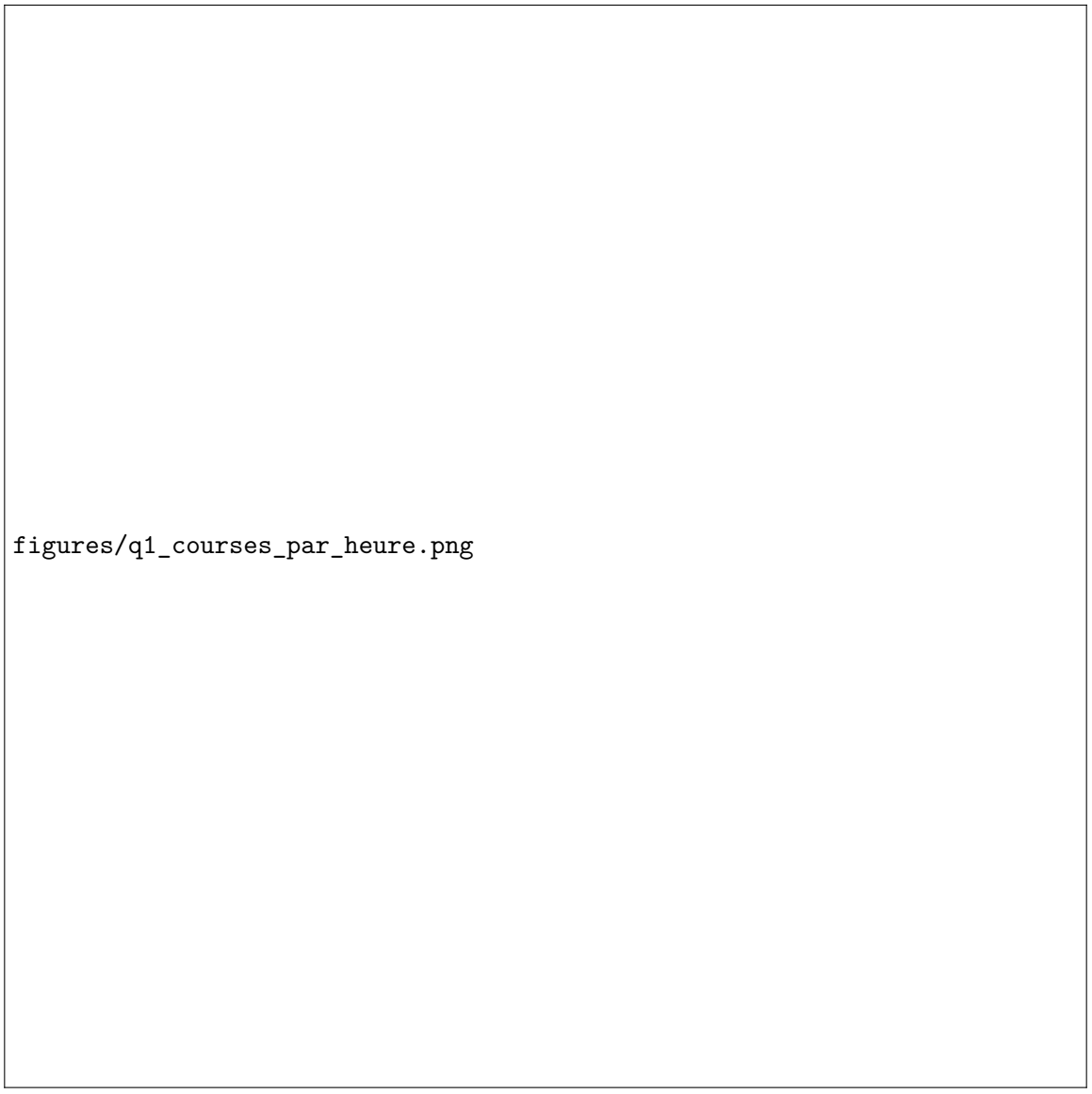


Figure 3 – Distribution des courses par heure de la journée (NYC, jan–mar 2024).

Pic à 18h (665 140 courses) — sortie de bureau. **Creux à 4h** (52 401 courses). Le revenu moyen est **maximal à 5h** (36,92 $) : ce sont les courses aéroportuaires tôt le matin (JFK → Manhattan).

6.2 Q2 — Top 10 zones les plus actives

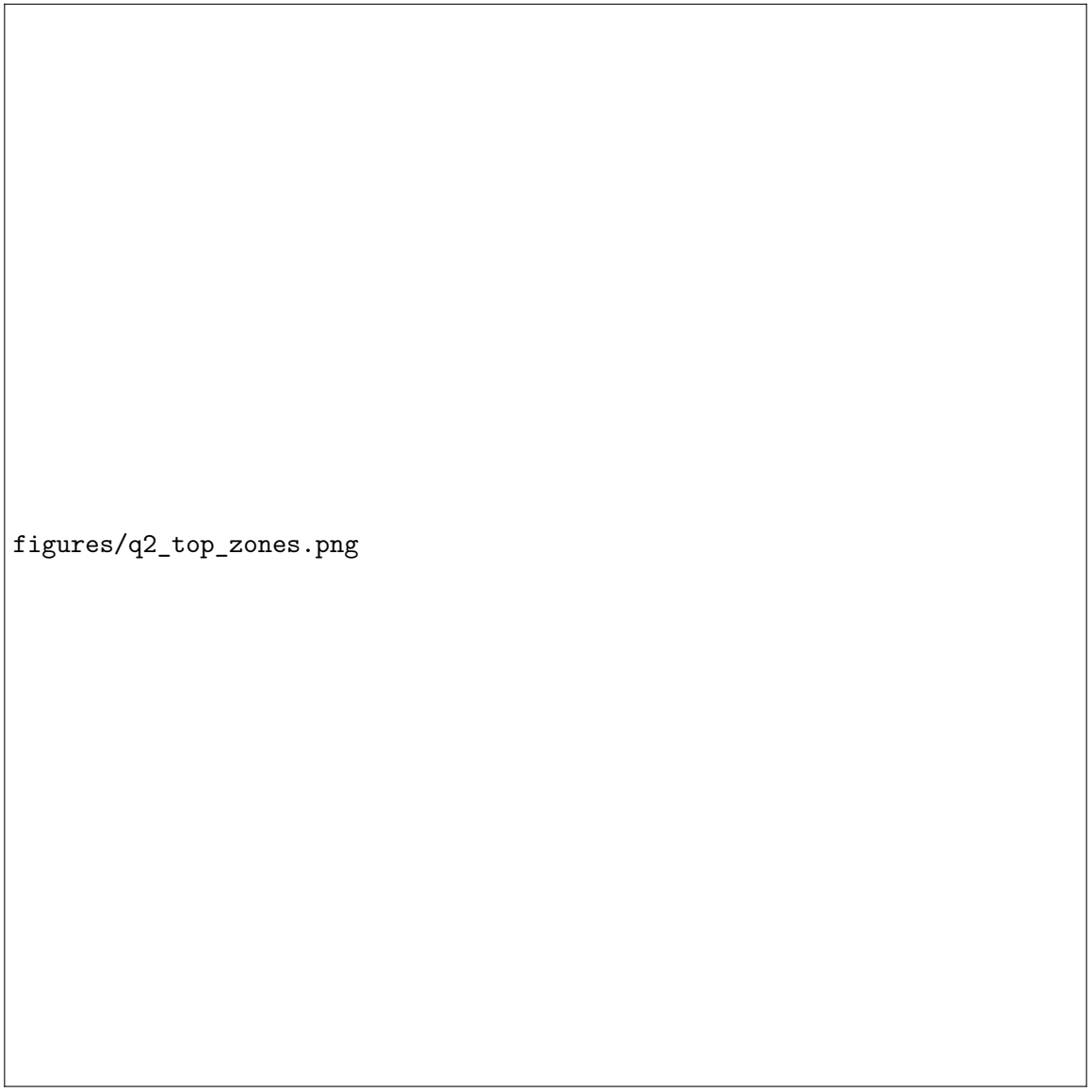


Figure 4 – Top 10 des zones de pickup par nombre de courses.

Table 4 – Top 4 des zones les plus actives

Rang	Zone	Borough	Courses
1	Midtown Center	Manhattan	441 059
2	Upper East Side South	Manhattan	429 087
3	Upper East Side North	Manhattan	406 164
4	JFK Airport	Queens	404 430

Manhattan domine largement l'activité, avec les deux aéroports (JFK, LaGuardia) en bonne position.

6.3 Q3 — Tip moyen par borough

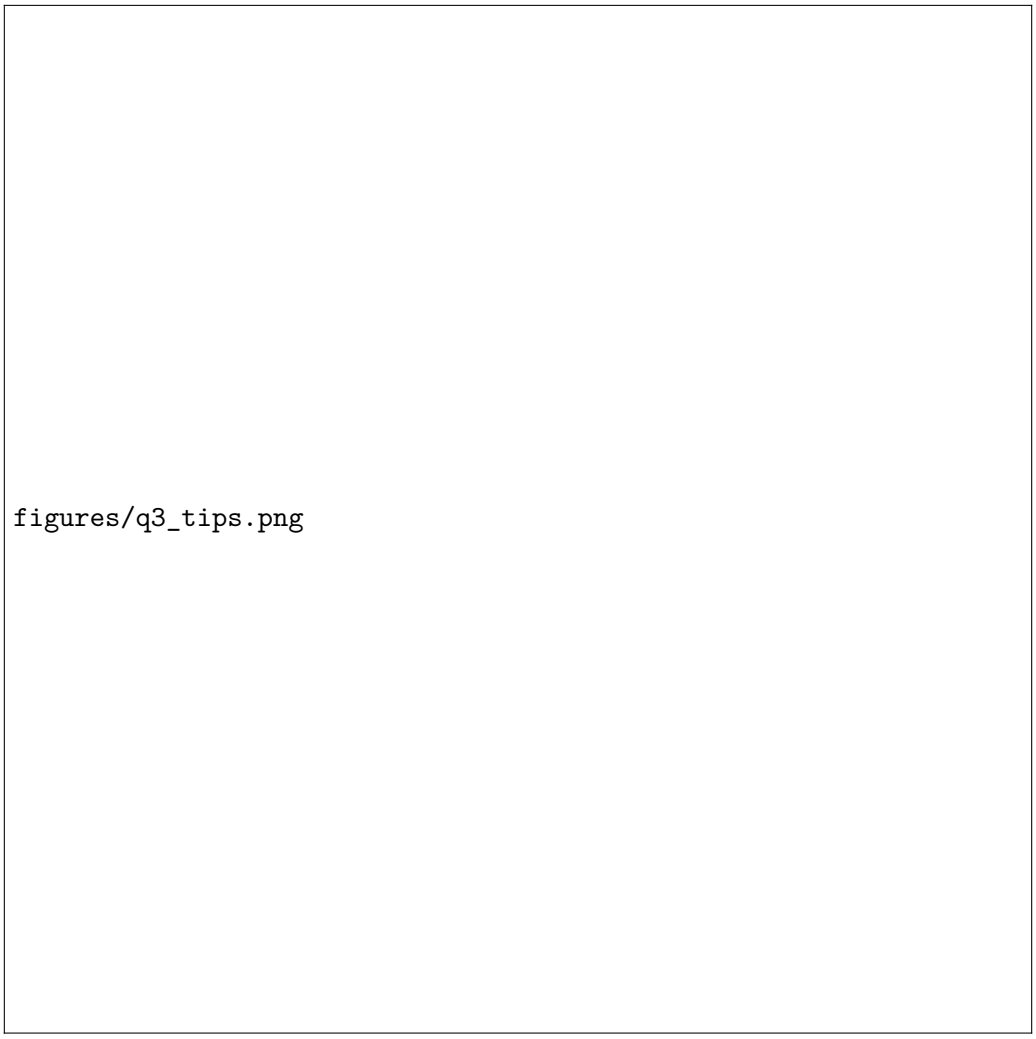


Figure 5 – Tip moyen par borough de départ.

Table 5 – Tip moyen par borough

Borough	Tip moyen	Nombre de courses
Manhattan	**20,73 %**	8 258 663
Queens	18,61 %	783 136
Brooklyn	5,94 %	86 336
Bronx	0,73 %	21 010

Manhattan et Queens concentrent les tips les plus élevés, tandis que le Bronx est très en dessous de la moyenne globale (10,3 %).

6.4 Q4 — Segmentation k-means des zones ($k = 5$)

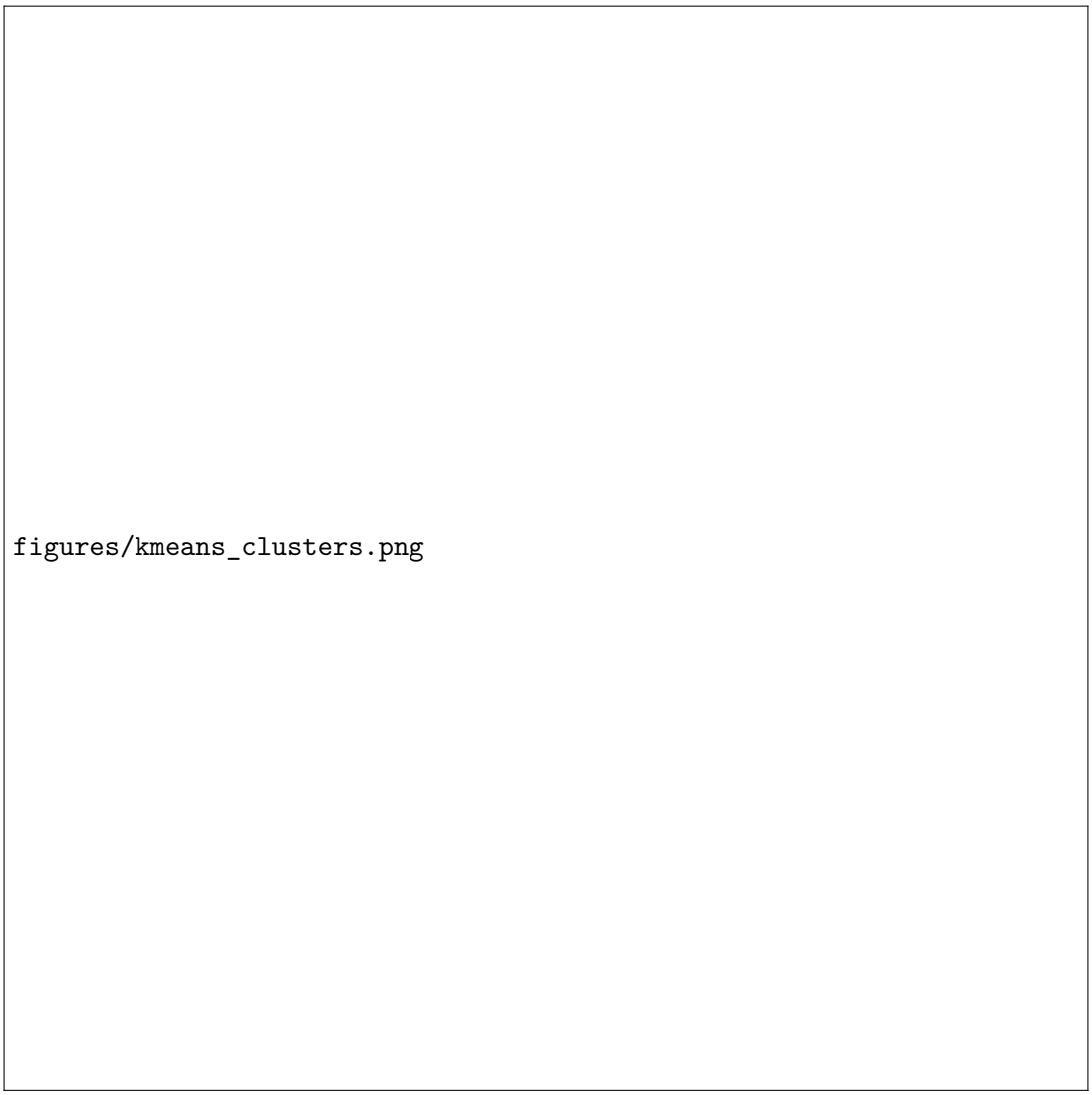


Figure 6 – Segmentation des zones de pickup par k-means ($k = 5$).

Table 6 – Profil des 5 clusters de zones

Cluster	Zones	Profil (courses / revenu)	Nom métier
3	2	341k / 74,6 $	Aéroports (JFK, LaGuardia)
4	13	313k / 23,2 $	Hyper-centre Manhattan
2	26	142k / 22,7 $	Manhattan résidentiel actif
0	69	8,9k / 31,0 $	Manhattan périphérique
1	10	5,1k / 63,4 $	Queens éloigné haute valeur

La segmentation fait émerger **5 profils distincts**, cohérents avec la géographie de New York : les aéroports se détachent par leur revenu élevé, Manhattan se divise entre hyper-centre et résidentiel, et Queens combine faible volume et haute valeur (courses longues vers Manhattan).

7 Conclusion

Ce qu'il faut retenir

— Sur **9,2 M de lignes**, Parquet est ×5,43 **plus compact** que CSV.
— Le partitionnement divise le temps de requête par ×3,34.
— Sur lecture partielle, le gain réel est de ×30 **à** ×50.
— Spark traite **9,2 M de lignes** là où Pandas sature à **2,9 M**.

Un bon pipeline Big Data n'est pas une question de matériel — c'est une question de conception. Passer de CSV à Parquet + partitionnement coûte *2 heures de code* et fait gagner *des heures de calcul par requête*. C'est l'investissement le plus rentable d'un projet data.

Limites et perspectives

— **Pas de streaming temps réel** : les données sont traitées en batch. Une extension avec Kafka + Spark Structured Streaming permettrait de traiter les flux en continu.
— **Pas de déploiement cloud** : tout tourne en local. Un déploiement S3 + EMR montrerait le passage à l'échelle.
— **Cache OS** : G2 et G3 sont gonflés par le cache. Une mesure sur disque froid (`drop_caches`) donnerait des chiffres plus conservateurs.
— **Pas de dashboard interactif** : les résultats sont dans le rapport et les PNG. Un dashboard Streamlit pourrait être ajouté en perspective.

Code source : https://github.com/IsmaelLebange/projet_nyc_taxi